KUPC Phase 8 Specification

Chat & Conversations

Version 1.0 · July 18, 2026 · Status: Draft for sprint planning · Detailed milestone steps

Document Control

Field	Value
Project	KUPC — Kathmandu University Placement Connect
Phase	8 — Chat & Conversations
Depends on	Phase 2 (Auth), Phase 3B (`conversations` / `messages` / `notifications` schema & repos), Phase 7 (matches exist from real swipe + reciprocation)
Feeds into	Later — richer notifications UX; optional realtime transport upgrades; admin moderation of abuse reports
Repos	kupc-backend + frontend (monorepo)
Audience	Backend engineers, frontend engineers, technical reviewers
References	PHASE_3B_DOCUMENTATION.md; PHASE_7_DOCUMENTATION.md; KUPC_Phase7_Specification.pdf; frontend/INTEGRATION.md; `conversations.repository.ts` / `messages.repository.ts`

1. Phase Goal

Turn a **match** into a durable 1:1 conversation: after a company reciprocates a student right-swipe, both parties can open a thread, exchange messages, and see read state. Replace the prototype Chat screen (hardcoded `MESSAGES` / `MATCHES` sidebar) and enable the Matches “Chat (Phase 8)” control.

Phase 7 answered: *How does interest become a match?*
Phase 8 answers: *How do matched parties talk about the role?*

2. Why This Phase Now

- Phase 7 shipped live Matches without messaging — the hiring loop stops at a badge.
- Phase 3B already provides `conversations` (1:1 per `match_id`), `messages`, RLS participant policies, and repositories (`createForMatch`, `listByConversation`, `markRead`).
- Seed data already creates ~25 conversations with opener messages — product paths must match that model (match-scoped threads only).
- Realtime WebSockets are **not** required for MVP; HTTP list + send + short-interval polling is enough for a campus demo, with SSE/WebSocket as an optional stretch.

3. Scope & Milestones

ID	Side	Topic	Outcome
B1	Backend	Chat module scaffold	Routes, Zod, DTOs, auth gates under `/conversations` (+ messages); stubs until B2/B3
B2	Backend	Ensure conversation + list mine	Get-or-create by `match_id`; list threads for student/company
B3	Backend	Send / list / mark-read messages	Participant-only send & history; optional mark read
B4	Backend	Match → conversation hook	Idempotent ensure on `POST /matches` (or document lazy-only)

ID	Side	Topic	Outcome
B5	Backend	Notifications (optional)	Insert `type=message` (and/or match) via `notificationsRepository`
B6	Backend	Swagger + test matrix	OpenAPI + `test:phase8` green
F1	Frontend	conversationsApi + messagesApi	Typed clients matching Phase 8 contracts
F2	Frontend	Matches → Chat entry	Enable Chat on match cards; deep-link `/app/chat?matchId=…` (or conversation id)
F3	Frontend	Live Chat page	Replace mock sidebar + thread with API list/send; polling refresh
F4	Frontend	Notifications polish (optional)	Surface unread / deep-link if B5 ships
F5	Frontend	Polish + docs	INTEGRATION.md + PHASE_8_DOCUMENTATION.md; whole-phase smoke

3.1 Out of scope

- Group chats, file attachments, voice/video calls
- Typing indicators / presence (may stay cosmetic mock)
- End-to-end encryption
- Admin message moderation tooling (reports table exists — later)
- Changing Phase 3 schema unless a documented gap is found (prefer repository extensions)
- Reopening Phase 7 swipe/match product rules

3.2 Suggested order

B1 → **B2** → **B3** → **B4** → **B6** on the backend (B5 optional in parallel after B3). Frontend may start **F1** as soon as B1 contracts are stable, then **F2–F3** after B2–B3, finish with **F5** (and **F4** only if B5 lands).

4. HTTP Contracts (illustrative)

All paths under `/api/v1`. Bearer JWT required. Responses use the existing envelope: `{ success, data, message, error }`. Wire naming remains **snake_case**.

4.1 Conversations

Method	Path	Auth	Description
GET	/conversations/me	Student or Company	List my threads: conversation + nested match job + counterparty summary + last message preview + unread count (if cheap).
POST	/conversations/ensure	Student or Company	Body: `{ match_id }`. Caller must be a participant on the match. Returns existing conversation or creates via `createForMatch` (idempotent).
GET	/conversations/:id	Participant	Conversation detail + counterparty card (optional if list is enough).

4.2 Messages

Method	Path	Auth	Description
GET	/conversations/:id/messages	Participant	Chronological messages. Query: `limit` / `before` (cursor optional; MVP may return full recent window).
POST	/conversations/:id/messages	Participant	Body: `{ content }`. Trimmed non-empty string; max length (e.g. 4000). `sender_id` = auth user.
POST	/conversations/:id/read	Participant	Mark peer messages as read (or mark all unread in thread). Returns `{ updated: n }` or conversation summary.

4.3 DTOs (illustrative)

- `ConversationDto` — id, match_id, created_at, nested `job` + counterparty (`student` or `company`), optional `last_message`, `unread_count`
- `MessageDto` — id, conversation_id, sender_id, content, sent_at, read_at
- `EnsureConversationBody` — `{ match_id }`
- `CreateMessageBody` — `{ content }`

4.4 Error codes

Code	HTTP	When
CONVERSATION_NOT_FOUND	404	Unknown id or not a participant
CONVERSATION_FORBIDDEN	403	Match exists but caller is not a party
MATCH_NOT_FOUND	404	ensure with invalid match_id
MESSAGE_NOT_FOUND	404	mark-read / lookup missing
INVALID_MESSAGE_PAYLOAD	400	Empty / oversized content
INVALID_CONVERSATION_PAYLOAD	400	Zod validation
NOT_IMPLEMENTED	501	Scaffold stubs until B2/B3

5. Locked Product Rules

- **One conversation per match** — unique `match_id`; never create a second thread for the same match.
- **Participants only** — student_id or company_id on the match (company user id = companies.id). Non-participants get 404/403 (prefer opaque 404 for enumeration).

• **No chat without a match** — Discover / Interest never open threads; Matches is the entry (or deep-link with match_id after ensure).

• **Service-role writes** — same as Phase 3B/7: feature modules use repositories + admin client; clients never insert via anon key.

• **Realtime MVP = polling** — Chat page polls messages every ~3–5s while focused (or on window focus). SSE/WebSocket is optional stretch, not a gate.

• **Eager vs lazy conversation** — **Lock:** support `POST /conversations/ensure` always; **also** call ensure (idempotent) from match create (B4) so seed-like data appears without opening Chat first.

5.1 Repository gaps to close

• Extend `conversationsRepository` with list-by-participant (student / company) joining matches — not present today (only `createForMatch` / finders).

• Optional: `messagesRepository.markAllUnreadForConversation` for peer messages; or loop `markRead`.

• Unread count: derive from messages where `sender_id ≠ me` and `read_at IS NULL`.

6. Milestone Details

Milestone B1 — Chat Module Scaffold & Contracts

Goal: Create `src/modules/conversations/` (messages nested under conversation routes or sibling module) with contracts, validation, and auth gates. Handlers may return `501 NOT_IMPLEMENTED` until B2–B3. **Depends on:** Phase 2 auth, Phase 3B repos.

1. Add constants, types, Zod schemas, errors, mappers, stub service, controller, routes.
2. Mount at `/api/v1/conversations`.
3. Static paths (`/me`, `/ensure`) before `/:id`.
4. Authorize STUDENT | COMPANY on all routes; participant checks land in B2/B3.
5. Add scaffold + mapper tests; `npm run test:phase8`.

Exit checklist

#	Criterion	Done when
1	Module exists	conversations module files present
2	Auth gates	No token → 401; wrong role → 403
3	Validation	Empty ensure/message body → 400
4	Stubs	Valid calls → 501 until B2/B3

Milestone B2 — Ensure Conversation + List Mine

Goal: Participants can open/list threads for their matches.

1. Implement ensure: load match → verify participant → `findByMatchId` or `createForMatch`.
2. Implement listMine: role-aware query of conversations via matches (extend repository as needed).
3. Map nested job + counterparty like Phase 7 MatchDto cards.

Exit checklist

#	Criterion	Done when
1	Ensure idempotent	Second ensure returns same conversation id
2	Ownership	Foreign match → 403/404
3	List	Student and company each see own threads

Milestone B3 — Send / List / Mark-Read Messages

Goal: Full message history and send for participants.

1. GET messages ordered by `sent_at` ascending.
2. POST message with auth user as sender; reject empty content.
3. Mark-read endpoint updates peer messages' `read_at`.
4. Reuse Express 5 `defineProperty` validate pattern for query params.

Exit checklist

#	Criterion	Done when
1	Send	201 MessageDto
2	List	Both participants see the same history
3	Non-participant	404/403

Milestone B4 — Match Create Hook

Goal: After Phase 7 `POST /matches` succeeds (including idempotent return), ensure a conversation row exists.

1. From `matchesService.create`, after insert/return, call get-or-create conversation (ignore unique races).
2. Do not fail the match if conversation ensure fails after match insert — log and rely on ensure endpoint; or fail closed if you prefer atomicity (document the choice in PHASE_8_DOCUMENTATION.md).
3. **Preferred lock:** best-effort ensure; Chat can still call ensure.

Exit checklist

#	Criterion	Done when
1	New match	Conversation row exists (or ensure recreates)
2	Idempotent match	No duplicate conversations

Milestone B5 — Notifications (Optional)

Goal: When a message is sent, notify the other participant (`type=message`, payload with conversation_id / match_id). Optionally notify on match create (`type=match`) if not done earlier.

Exit checklist

#	Criterion	Done when
1	Insert	Row in notifications for the peer
2	Skip if deferred	Document as out of Phase 8 MVP

Milestone B6 — Swagger, Hardening & Test Matrix

1. Document all §4 paths in `swagger.ts`.
2. Add `phase8.swagger` + `phase8.matrix` tests; `test:phase8` script.
3. Keep `test:phase7` green.

Exit checklist

#	Criterion	Done when
1	OpenAPI	Conversation/message paths at /api/docs
2	test:phase8	Full suite green

Milestone F1 — conversationsApi & messagesApi

1. Add types + clients under `frontend/src/lib/api/`.
2. Map Phase 8 error codes in `errorMessages.ts`.

Exit checklist

#	Criterion	Done when
1	Clients export	Importable from lib/api
2	Types compile	tsc clean

Milestone F2 — Matches → Chat Entry

1. Replace disabled Chat button with navigation to Chat using `match_id` (call ensure on open if needed).
2. Empty Matches still CTA to Discover / Interest as today.

Exit checklist

#	Criterion	Done when
1	Entry	From Matches, Chat opens the right thread
2	No match	Cannot invent a conversation id

Milestone F3 — Live Chat Page

1. Replace prototype ChatPage: sidebar from `GET /conversations/me`; thread from messages list; send via POST.
2. Poll while tab focused; ErrorBanner on failure.
3. Remove hardcoded MESSAGES / MATCHES usage from Chat (keep nowhere else unless documented).

Exit checklist

#	Criterion	Done when
1	Send/receive	Both seed accounts can converse on a match
2	Mock removed	No hardcoded message array for live Chat

Milestone F4 — Notifications Polish (Optional)

1. If B5 ships: wire Notifications page subset or badge to `notificationsRepository` list API (may need a thin notifications module if none exists).
2. Deep-link notification → Chat thread.

Milestone F5 — Polish & Documentation

1. Update `frontend/INTEGRATION.md` live-vs-mock (Chat live; dashboards still mock).
2. Write `kupc-backend/documentation/PHASE_8_DOCUMENTATION.md` as milestones ship.
3. Manual smoke: match → both open Chat → exchange messages → refresh persists.

Exit checklist

#	Criterion	Done when
1	Docs	INTEGRATION + PHASE_8_DOCUMENTATION
2	Smoke	End-to-end path works on seed accounts

7. Phase 8 Exit Checklist (Whole Phase)

#	Criterion	Owner
1	B1–B3 (and B4/B6) complete; npm run test:phase8 green	Backend
2	Swagger documents conversation + message endpoints	Backend
3	Ensure conversation is idempotent per match	Backend
4	Only match participants can list/send messages	Backend
5	Match create ensures conversation (or ensure recovers)	Both
6	Matches Chat entry opens the correct thread	Frontend
7	Chat page no longer uses hardcoded MESSAGES for data	Frontend
8	Messages persist across refresh for both parties	Both
9	INTEGRATION.md + PHASE_8_DOCUMENTATION.md written	Both
10	test:phase7 still green; frontend tsc clean	Both

7.1 Manual smoke script

1. Login seed.student.001 → Discover → Like an open job owned by an approved seed company.
2. Login that company → Interest → Match the student.
3. Both accounts → Matches → Chat → ensure thread opens (same conversation).
4. Student sends a message; company refreshes / polls → sees it; reply.
5. Reload Chat → history still present.
6. Non-participant JWT must not read that conversation id.
7. Pending company still blocked from Match (Phase 7); no chat without match.

8. After Phase 8

Optional upgrades: SSE/WebSocket push, richer notification center, attachment uploads, and admin tools on `reports`. Do not expand chat into multi-party or job-broadcast channels without a new phase.

End of KUPC Phase 8 Specification · Generated July 18, 2026 · Implement milestones in order; update PHASE_8_DOCUMENTATION.md as each milestone ships.